

Capitolo 9 - Memoria Centrale

Status Done

La memoria centrale (RAM) è essenziale per il funzionamento di un computer moderno. Consiste in un grande vettore di byte, ognuno con il proprio indirizzo. La CPU esegue le istruzioni caricate da questa memoria e vi legge/scrive i dati.

Il punto cruciale è che:

- la CPU preleva le istruzioni basandosi sul contatore di programma.
- Le istruzioni eseguite possono a loro volta causare ulteriori operazioni di lettura (load) o scrittura (store) in memoria.
- L'unità di memoria vede semplicemente un flusso di indirizzi, senza preoccuparsi di come siano stati generati (se sono istruzioni o dati).

Protezione dell'hardware

Per garantire la corretta esecuzione del sistema e impedire che un programma utente danneggi il SO o altri programmi, è fondamentale separare e proteggere le aree di memoria.

Il meccanismo di protezione hw si basa su due registri fondamentali:

- **Registro Base (Base Register):** contiene l'indirizzo fisico iniziale più piccolo a cui un processo può accedere.
- **Registro Limite (Limit Register):** specifica la dimensione dell'intervallo di indirizzi logici validi per il programma.

Quando un programma utente genera un indirizzo in modalità utente, la CPU lo confronta con questi due registri.

- L'indirizzo deve essere maggiore o uguale al valore del registro base.
- L'indirizzo deve essere minore della somma del registro base e del registro limite (quindi entro i limiti dell'area assegnata).
- Se la verifica fallisce, viene generata un'eccezione (**trap**) che restituisce il controllo al SO, trattando l'evento come un errore fatale.

Solo il SO (eseguito in modalità kernel) può caricare e modificare i valori dei registri base e limite, in quanto è un'operazione che richiede un'istruzione privilegiata.

Associazione degli indirizzi

Il processo di associazione delle istruzioni e dei dati agli indirizzi di memoria può avvenire in diverse fasi:

1. **Fase di compilazione:** se si conosce a priori la posizione esatta in memoria (indirizzo iniziale **R**), si può generare codice assoluto. Se l'indirizzo dovesse cambiare, il codice andrebbe ricompilato.
2. **Fase di caricamento:** se la posizione non è nota al momento della compilazione, si genera codice rilocabile. L'associazione finale degli indirizzi viene rimandata al momento del caricamento. Se l'indirizzo iniziale cambia, basta ricaricare il codice.
3. **Fase di esecuzione (Associazione dinamica):** se il processo può essere spostato in memoria durante la sua esecuzione, l'associazione deve essere ritardata a questa fase. Questa tecnica richiede un supporto hw specifico (come la **MMU - Memory Management Unit**).

Quando l'associazione avviene nella fase di esecuzione, gli indirizzi generati dalla CPU (chiamati **indirizzi logici** o **indirizzi virtuali**) sono diversi dagli indirizzi visti dall'unità di memoria (chiamati **indirizzi fisici**).

Unità di gestione della memoria (MMU)

È un dispositivo hw che esegue la mappatura dinamica degli indirizzi logici in indirizzi fisici al momento dell'esecuzione.

In uno schema base, questo si ottiene utilizzando un **registro di rilocalizzazione** (che corrisponde al registro base):

$$\text{Indirizzo Fisico} = \text{Indirizzo Logico} + \text{Contenuto del Registro di Rilocalizzazione}$$

In pratica, il programma utente lavora solo con indirizzi logici, e solo quando un indirizzo è inviato all'hw di memoria, la MMU esegue la traslazione sommandogli il valore del registro di rilocalizzazione. Il programma utente non vede mai gli indirizzi fisici reali.

Caricamento dinamico

Un programma o un processo può essere suddiviso in routine rilocabili, non tutte necessarie contemporaneamente.

- Con il caricamento dinamico, una routine viene caricata in memoria solo quando viene invocata.
- Tutte le routine non utilizzate possono rimanere su disco.

- Questa tecnica riduce lo spazio di memoria richiesto e consente di gestire programmi molto grandi. Non richiede uno speciale supporto dal SO, ma necessita di un'interfaccia utente che si occupi del caricamento.

Collegamento dinamico

Con il collegamento statico, le librerie di sistema sono collegate al programma al momento della compilazione o del caricamento. Questo comporta:

- L'inclusione di una copia della libreria nel file immagine del programma, sprecando spazio su disco e memoria.

Il **collegamento dinamico** (o collegamento condiviso) rimanda gran parte del lavoro di collegamento all'esecuzione del programma.

- Al programma viene fornito solo un piccolo **stub** per ogni routine di libreria.
- Quando lo stub viene eseguito per la prima volta, carica la libreria necessaria in memoria (se non è già lì), sostituisce se stesso con l'indirizzo di inizio della routine e la esegue.
- Questa tecnica è particolarmente utile con le librerie di sistema, che possono essere condivise tra più processi. Ogni processo utilizza un indirizzo virtuale per accedere alla libreria, ma l'indirizzo fisico corrisponde alla stessa copia in memoria.

Allocazione contigua della memoria

L'allocazione contigua della memoria è un metodo fondamentale di gestione della memoria che impone che la memoria fisica assegnata a ciascun processo sia un'unica regione contigua e singola.

Struttura della memoria

La memoria centrale è generalmente divisa in due partizioni:

- una per il SO residente;
- una o più per i processi utente.

Per la protezione e la rilocalizzazione di ogni processo utente, vengono utilizzati i registri Base e Limite. L'obiettivo è mantenere più processi utente in memoria contemporaneamente per supportare la multiprogrammazione.

Gestione dello spazio libero

Quando la memoria è allocata e rilasciata dinamicamente, si formano degli spazi liberi non allocati, chiamati buchi (holes).

- Quando un processo entra, il sistema cerca un buco libero abbastanza grande da soddisfare la sua richiesta.
- Quando un processo termina, rilascia il suo blocco, che viene unito a eventuali buchi adiacenti per formare un buco più grande.

Il problema della frammentazione

L'allocazione contigua, specialmente quella a partizione variabile, è afflitta dal problema della **frammentazione**.

Frammentazione esterna

- Si verifica quando lo spazio totale di memoria libera esiste, ma è suddiviso in piccoli buchi non contigui, e nessuno di essi è abbastanza grande da soddisfare una nuova richiesta.
- La "regola del 50 per cento" empirica suggerisce che, in condizioni ideali, circa un terzo dello spazio totale disponibile potrebbe rimanere inutilizzato a causa della frammentazione esterna.
- **Soluzione:** l'unico rimedio generale è la **compattazione** (compacting), che sposta i processi in memoria per raggruppare tutti i buchi sparsi in un unico grande blocco libero. Tuttavia, è possibile solo se la rilocalizzazione è dinamica e si effettua nella fase d'esecuzione, in quanto richiede solo lo spostamento del programma e dei dati.

Frammentazione interna

- Si verifica quando la memoria allocata a un processo è leggermente maggiore della memoria effettivamente richiesta.
- Il frammento di memoria inutilizzato si trova all'interno di una partizione allocata e non può essere utilizzato da nessun altro processo.
- Questo problema è comune quando l'allocazione avviene in unità di dimensione fissa (come i blocchi di allocazione o le pagine).

Strategie di allocazione dello spazio libero

Quando è disponibile più di un buco libero in grado di soddisfare una richiesta di memoria, il SO deve decidere quale scegliere. I criteri di selezione più comuni sono:

- **First-Fit** (Prima Occorrenza)
 - Alloca il primo buco trovato che sia abbastanza grande.

- **Vantaggio:** molto veloce perché smette di cercare immediatamente.
- **Best-Fit** (Migliore Occorrenza)
 - Alloca il buco libero più piccolo che soddisfi la richiesta.
 - **Vantaggio:** tende a lasciare buchi grandi per richieste future.
 - **Svantaggio:** genera la massima frammentazione esterna (molti piccoli buchi residui) e richiede una scansione completa.
- **Worst-Fit** (Peggior Occorrenza)
 - Alloca il buco libero più grande disponibile.
 - **Vantaggio:** il buco residuo lasciato è grande e probabilmente utile per le prossime richieste.
 - **Svantaggio:** di solito più lento e meno efficiente di First-Fit e Best-Fit.

Le simulazioni indicano che first-fit e best-fit sono generalmente più efficaci di worst-fit in termini di tempo e di utilizzo dello spazio.

Paginazione

La paginazione è uno schema di gestione della memoria che permette che lo spazio di indirizzamento fisico di un processo sia non contiguo. Questo metodo è cruciale perché evita la frammentazione esterna e la necessità della costosa operazione di compattazione.

Metodo di base

L'implementazione della paginazione richiede la cooperazione tra il SO e l'HW del computer.

- **Memoria fisica (RAM):** suddivisa in blocchi di dimensione fissa chiamati **frame**.
- **Memoria logica (spazio degli indirizzi del processo):** suddivisa in blocchi di pari dimensione chiamati **pagine**.
- **Tabella delle pagine (page table):** una struttura dati che esegue la mappatura. Mantiene la corrispondenza tra i numeri di pagina di un processo e i numeri di frame fisici disponibili.

Per tradurre un indirizzo logico generato dalla CPU in un indirizzo fisico, la MMU esegue questi passaggi:

1. L'indirizzo logico è diviso in **numero di pagina (p)** e **offset (d)**.
2. Il numero di pagina **p** è usato come indice nella **Tabella delle pagine** per trovare il **numero di frame (f)** corrispondenti.
3. Il numero di frame **f** viene combinato con l'**offset d** per ottenere l'**indirizzo fisico**.

La dimensione di pagina e frame è definita dall'HW ed è tipicamente una potenza di 2 per semplificare la traduzione degli indirizzi.

Paginazione e frammentazione interna

La paginazione elimina la frammentazione esterna perché qualsiasi frame libero può essere assegnato a un processo. Tuttavia, può causare **frammentazione interna**. Poiché i frame sono assegnati come unità intere, l'ultimo frame assegnato a un processo potrebbe non essere completamente utilizzato (es. con pagine da 2048 byte, un processo che richiede 1086 byte spreca 962 byte).

Supporto hardware alla paginazione

La tabella delle pagine deve essere accessibile per ogni riferimento alla memoria. Per un sistema a 32 bit, con pagine da 4 KB, la tabella può contenere fino a 1 milione di elementi.

Archiviazione della tabella delle pagine

- **Posizione:** la tabella delle pagine è tipicamente memorizzata in **memoria centrale (RAM)**.
- **Overhead:** per ogni accesso alla memoria logica da parte di un programma, sono necessari due accessi alla memoria fisica:
 1. la CPU deve prima leggere la tabella delle pagine (in RAM),
 2. poi l'istruzione (nel frame in RAM).
- **Registri CPU:** un **puntatore (Page-Table Base Register - PTBR)** memorizzato nel blocco di controllo del processo (PCB) punta alla posizione iniziale della tabella delle pagine del processo in memoria.

Translation Look-Aside Buffer (TLB)

Per evitare l'overhead del doppio accesso alla memoria, si utilizza una **cache hardware specializzata** ad alta velocità chiamata **Translation Look-Aside Buffer (TLB)**.

- **Funzionamento:** il TLB è una memoria associativa che memorizza un sottoinsieme di voci della tabella delle pagine (coppie <numero di pagina, numero di frame>).

1. La MMU cerca il numero di pagina del TLB: se la voce è presente (**TLB Hit**), il frame è ottenuto velocemente, senza andare in RAM.
2. Se non è presente (**TLB Miss**), l'indirizzo deve essere letto dalla tabella delle pagine in RAM, il frame viene letto, e l'informazione viene aggiunta al TLB per accessi futuri

Prestazioni del TLB

L'efficacia del TLB è misurata dal **Tasso di Successo (Hit Ratio, α)**: la percentuale di volte in cui una voce è trovata nel TLB.

- **Tempo di accesso effettivo (Effective Access Time - EAT)**: migliora significativamente grazie al TLB.
- **Formula (EAT)**: siano α il tasso di successo del TLB e τ il tempo impiegato per cercare nel TLB.

$$EAT = (1 + \tau)\alpha + (2 + \tau)(1 - \alpha)$$

(Assumendo che τ sia trascurabile rispetto al tempo di accesso alla memoria M)

$$EAT = \alpha \times M + (1 - \alpha) \times 2M$$

Un alto tasso di successo è cruciale; ad esempio, se $\alpha = 99\%$ e il tempo di accesso alla memoria è 100 nanosecondi, l'EAT scende drasticamente (da 200 ns senza TLB a 101 ns con TLB).

Protezione della memoria con paginazione

La paginazione fornisce meccanismi di protezione avanzati:

- **Bit di validità/Non validità (Valid-Invalid Bit)**: a ogni voce della tabella delle pagine è associato un bit per indicare se la pagina è valida (risiede nello spazio degli indirizzi logici del processo e ha un frame assegnato) o non valida (non fa parte dello spazio degli indirizzi logici).
- **Bit di protezione (Lettura/Scrittura/Esecuzione)**: controlla il tipo di accesso consentito al contenuto di un frame. Ad esempio, si possono segnare le pagine contenenti codice come di sola lettura.
- **Registri di lunghezza della tabella delle pagine (Page-Table Length Register - PRLR)**: contiene la lunghezza della tabella delle pagine, consentendo di verificare che l'indirizzo logico generato sia inferiore alla dimensione della tabella.

Pagine condivise

La paginazione consente la condivisione di codice e dati tra più processi, con un notevole risparmio di memoria:

- **Condivisione di Codice (Code Sharing)**: il codice che è rientrante (o puro) può essere condiviso e può risiedere in un singolo set di frame fisici in RAM. Il codice rientrante è un codice che non modifica se stesso durante l'esecuzione ed è quindi di sola lettura.
 - Esempio: le librerie di sistema possono essere condivise tra tutti i processi che le utilizzano. Ciascun processo ha la sua copia della tabella delle pagine che punta agli stessi frame fisici.
- **Sezione dati private**: ogni processo deve mantenere le proprie sezione dati, stack e heap private (non rientranti), ciascuna mappata a frame fisici distinti.

Strutture della tabella delle pagine

Quando gli spazi di indirizzamento logico diventano molto grandi, la tabella delle pagine può diventare enorme. Se ogni voce occupa 4 byte e lo spazio logico è di 2^{64} byte, si avrebbero tabelle di pagine gigantesche, impossibili da allocare in modo contiguo in RAM.

Per affrontare il problema della dimensione, i SO moderni utilizzano diverse tecniche per strutturare la tabella delle pagine in modo più efficiente:

1. **Paginazione Gerarchica (Multi-livello)**
2. **Tabella Hash delle Pagine Invertita (Inverted Page Table)**
3. **Segmentazione con Paginazione (Hashed Page Tables)**

Paginazione gerarchica

La paginazione gerarchica (spesso chiamata tabella delle pagine multilivello) è la soluzione più comune per affrontare tabelle di pagine enormi.

L'idea è: invece di avere una singola tabella delle pagine, la si suddivide in pezzi più piccoli.

- Si applica la paginazione alla tabella delle pagine stessa.

- Se la tabella delle pagine è troppo grande, la si considera come un oggetto che, a sua volta, può essere paginato.

L'indirizzo logico non viene diviso solo in **Pagina** e **Offset**, ma in:

- **Indice della tabella esterna:** indice per trovare la tabella di secondo livello (la vera e propria tabella di pagine).
- **Indice della tabella interna:** indice per trovare il frame fisico.

Paginazione a due livelli

In un sistema a 32 bit (con pagine da 4 KB), la paginazione a due livelli divide l'indirizzo logico in tre parti:

1. **Indice della tabella esterna (Outer Page Table Index - P_1):** serve per trovare la voce nella tabella principale (detta **Tabella delle Pagine Esterne**).
2. **Indice della Tabella Interna (Inner Page Table Index - P_2):** serve per trovare la voce nella tabella secondaria (detta **Tabella delle Pagine Interne**), che è la vera e propria tabella delle pagine.
3. **Offset (d):** l'offset all'interno della pagina fisica.

Meccanismo di traduzione:

1. La CPU utilizza P_1 per trovare l'indirizzo della tabella interna corretta.
2. Quindi utilizza P_2 per indicizzare la tabella interna e trovare il frame fisico.
3. Combina il frame con l'offset d

Svantaggio: per trovare un indirizzo fisico sono necessari tre accessi alla memoria (uno per la tabella esterna, uno per la tabella interna e uno per i dati/istruzioni), a meno che non si utilizzi il TLB per caching.

Tabella Hash delle pagine invertita

La paginazione gerarchica risolve l'allocazione, ma ogni processo deve avere la sua tabella di pagine, che può essere grande. La tabella Hash delle Pagine Invertita (Inverted Page Table - IPT) risolve il problema della dimensione ($\Rightarrow$ riduce lo spazio totale richiesto per tutte le tabelle).

- Invece di avere una tabella per ogni processo (che mappa la pagina logica al frame fisico), si ha una sola tabella per tutto il sistema (la IPT).
- La IPT ha un numero di voci pari al numero di frame fisici in RAM.
- Unavoce della IPT in posizione i indica:
 1. L'ID del Processo (PID) che sta usando quel frame i .
 2. Il numero di pagina logica del processo che risiede in quel frame .
 3. Il numero di frame fisico in cui risiede (implicito dalla posizione nella tabella).

Meccanismo di traduzione

Quando la CPU genera un indirizzo logico (PID, Pagina p , offset d):

1. La MMU esegue una ricerca nella IPT (spesso usando una funzione di **hash**) per trovare una voce che corrisponda sia al PID che al numero di pagina p .
2. Se si trova una corrispondenza, l'indice della voce trovata è il numero di frame f in cui risiede la pagina.
3. L'indirizzo fisico è f combinato con l'offset d .

Vantaggio: la dimensione della tabella delle pagine non dipende più dalla dimensione dello spazio di indirizzamento logico, ma dalla dimensione della RAM fisica.

Svantaggio: la traduzione dell'indirizzo è molto più complessa e lenta perché richiede la ricerca per corrispondenza nella IPT. Anche in questo caso, il TLB è fondamentale per mitigare la lentezza.

Spazi di indirizzamento condivisi

L'uso di pagine condivise è facilitato dalla paginazione gerarchica, dove diversi processi possono avere voci separate nelle loro tabelle di pagine che puntano allo stesso set di frame fisici.

- Se due processi devono usare lo **stesso codice in sola lettura** (ad esempio, una libreria di sistema), non c'è bisogno di caricarla due volte in RAM.
- Le loro **tabelle di pagine individuali** (o la singola IPT, a seconda dello schema) possono semplicemente puntare allo **stesso set di Frame Fisici** che contengono il codice condiviso.

Tabella delle pagine per architetture a 64 bit

Le architetture a 64 bit offrono uno spazio di indirizzamento logico teoricamente enorme (2^{64} byte). Anche se i SO attuali non usano l'intero spazio (spesso 2^{48} o 2^{52}), le tabelle delle pagine a 64 bit sono così grandi da richiedere soluzioni avanzate:

Paginazione Gerarchica Estesa

- Un sistema a **quattro livelli** di paginazione può essere necessario per mappare uno spazio di indirizzamento a 64 bit. Questo porta a quattro accessi alla memoria (più uno per i dati) senza l'uso del TLB.

Hashed Page Tables

- Simile alla IPT, ma spesso adattata per affrontare gli spazi di indirizzamento sparsi. Utilizzano una funzione hash per indirizzare una catena (chain) di voci, riducendo l'area di ricerca e mantenendo la dimensione della tabella gestibile.

In sintesi, l'uso di **Paginazione Gerarchica** (multilivello) o di **Tabella Hash Invertita** è obbligatorio per gestire l'enorme spazio di indirizzamento virtuale delle architetture moderne, anche se rende la traduzione degli indirizzi più complessa e dipendente da una cache veloce come il **TLB**.